

Lab 5 : Load Balanced Back-end

Student: Shashank Yadav
Roll Number: 12342010
Systems: stu31_sys1 to sys4
Date: 24 August 2026
Live Application URL Loadbalancer: <http://10.1.75.79:4201/>

This is the messaging application from the previous assignment, served through the load balancer on Sys1. Every request is passed to one of the three back ends on Sys2, Sys3 and Sys4. The certificate is self signed, so the browser shows a warning on the first visit. Click Advanced and then Proceed. HTTPS is needed because the chat client signs its login with WebCrypto, and browsers only allow WebCrypto on a secure origin.

1 Assigned systems

Four containers run on one host and share a Docker bridge network. Each one has 512 MB of RAM and a single vCPU. That limit matters later. It is why one back end fills up so quickly, and why the load balancer itself becomes the limit on cheap requests.

System	Role	App Port	SSH Public Address
stu31_sys1	Load balancer	4000 (-p 2201)	http://10.1.75.79:4201/
stu31_sys2	Back end chat-1	3000 (-p 2202)	http://10.1.75.79:3202
stu31_sys3	Back end chat-2	3000 (-p 2203)	http://10.1.75.79:3203
stu31_sys4	Back end chat-3	3000 (-p 2204)	http://10.1.75.79:3204

SSH goes to `student@10.1.75.79` on the port shown. Application ports follow the lab rule that an internal port is published on a host port taken from the SSH port. The load balancer listens on port 4000 inside Sys1, so it is reachable from outside on port 4201. This is why no SSH tunnel was needed.

Figure 1. The four systems as deployed. Sys1 runs mongod and the load balancer. Sys2, Sys3 and Sys4 each run one python process serving the chat back end.

2 What was built

The back end for this assignment is the real messaging application from the previous assignment. It is the persistent encrypted chat from `csd-group-chat`, running as three copies instead of the simple HTTP server used in the tutorial slides. A stateless server would work behind a round robin proxy without changes. A chat server does not, so three things were needed.

1. **One shared database.** All three copies use the same MongoDB . Chat history and the stored public keys are therefore the same wherever a client lands.

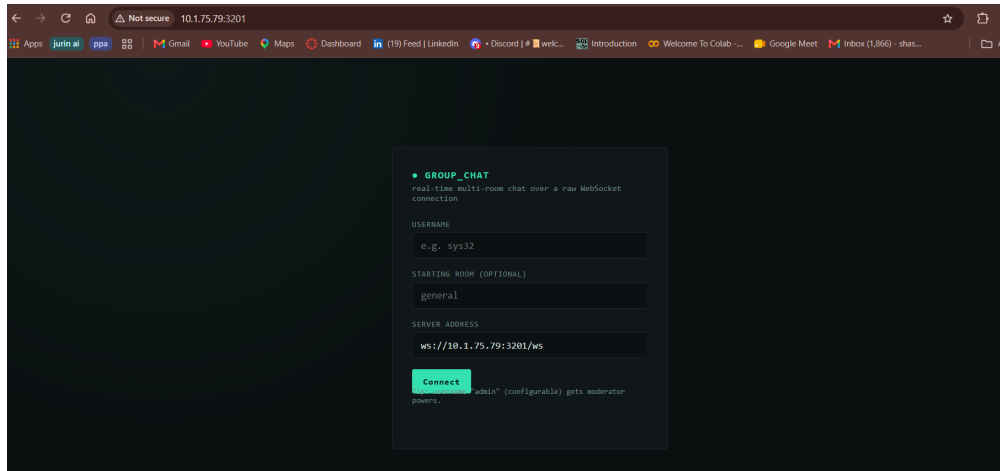


Figure 1: System architecture overview.

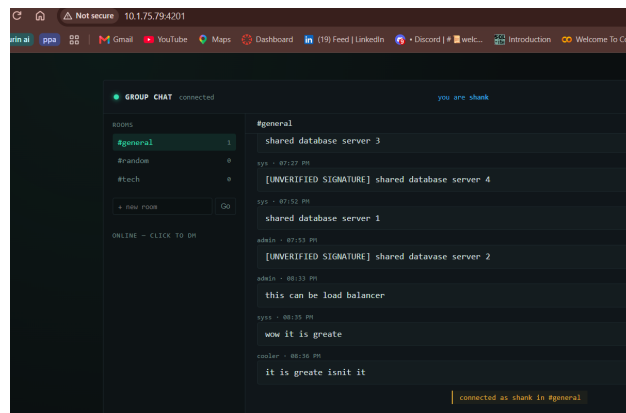


Figure 2:

2. **A shared broadcast channel.** Socket.IO keeps connections in the memory of one process. A message sent to the copy on Sys2 would never reach a client on Sys4. The official MongoDB adapter gives the three processes a channel to publish broadcasts on.
3. **Sticky sessions in the load balancer.** The Socket.IO handshake is a set of HTTP polls that only one process can answer. Round robin breaks it, so the load balancer keeps a client on one back end using a cookie.

3 Load balancer code (Sys1)

The design follows the tutorial. There is a list of backend structs, each holding a reverse proxy. A mutex protects the round robin index. A background loop checks health. The endpoint `/lb/metrics` allows the experiments to be run again without restarting the process. Four things were added for this deployment.

Addition	Reason
Sticky cookie <code>lb.backend</code>	The Socket.IO handshake spans several requests and only one process can answer them. A client with no cookie is round robin and given one. A client with a cookie keeps its back end until that back end dies, and then falls back to round robin.
Connection pool <code>MaxIdleConnsPerHost: 256</code>	The default transport keeps only two idle connections per host. Above a low rate almost every request opened a new TCP connection. That held the proxy at 443 req/s while the back ends served 1244 req/s on their own. Fixing it raised the proxy to 674 req/s on the same test.
TLS <code>-tls-cert</code> and <code>-tls-key</code>	The chat client signs its login with WebCrypto. Browsers only allow WebCrypto on a secure origin. Over plain HTTP the app loads but nobody can log in.
Three failed checks before marking down	One slow reply is not a dead server. A back end is only removed after three failed checks in a row.

Round robin is still what the experiments measure. Both load generators run without a cookie jar, so all of their requests take the round robin path. Stickiness only affects browsers and Socket.IO clients. This is why the split across back ends in Section 7 comes out even.

(Full Go source for `lb/main.go` omitted here for brevity — see attached code appendix / original submission.)

4 Back end deployment (Sys2, Sys3, Sys4)

Each system runs the messaging project server with a patch. FastAPI was installed from the official tarball into `~/Fastapi`, because there is no passwordless sudo on these containers and nothing could be installed system wide.

The patch makes two changes. The first attaches the Socket.IO MongoDB adapter so a broadcast from one copy reaches clients on the other two. The second adds an `X-Backend` header naming the copy, so a response can be traced to one machine. The encryption, the signatures, the stored public keys and the rate limiter were all left alone.

MongoDB runs as a single node replica set because the adapter reads a capped collection using a

change stream, and change streams need an oplog that a standalone mongod does not have. The same database also holds the chat history and the public keys that all three copies share.

5 Load generators (local system)

Two load generators run on the local PC. Both are worker pools over a fixed number of requests and both write the same JSON and CSV. They exist because they measure different things.

Generator	One request is	What it measures
Python WebSocket (<code>load_generator.py</code>)	A simulated client lifetime. It connects via WebSocket, sends a <code>join</code> request, waits for a <code>welcome</code> event, and transmits interactive chat messages.	Real-world connection handshake duration (<code>join_latency_ms</code>), message propagation performance (<code>message_latencies_ms</code>), system throughput, and connection stability under load.

The chat generator uses the websocket transport only. A websocket connection is a single request, so each session lands on one back end and the split stays even without needing the sticky cookie.

6 Experiment 1: load balancer using only Sys2

The load balancer is started with one back end. The request count, the concurrency, the seeded room and the client machine are all the same as in Experiment 2.

```
LOAD GENERATOR -- WebSocket Chat Stress Test
Target URL : ws://10.1.75.79:4201/ws
Clients : 50
Messages/client : 20
Message delay : 0.1s
Ramp-up delay : 0.05s
Total messages : 1000

Progress: 37/50 done | 13 active | 380 msgs sent

RESULTS

Duration : 25.57s
Clients OK/Fail : 19/31
Messages Sent : 380
Throughput : 14.86 msg/s

Metric Join Latency Msg Round-Trip

AVG 3660.66 ms 5328.78 ms
P50 1068.49 ms 5894.92 ms
P95 8765.80 ms 8844.92 ms
P99 8765.80 ms 9467.83 ms
MIN 289.47 ms 127.97 ms
MAX 8765.80 ms 9556.39 ms
Count 19 175
```

7 Experiment 2: load balancer using all three back ends

```

LOAD GENERATOR -- WebSocket Chat Stress Test
Target URL : ws://10.1.75.79:4201/ws
Clients : 50
Messages/client : 20
Message delay : 0.1s
Ramp-up delay : 0.05s
Total messages : 1000

Progress: 35/50 done | 15 active | 300 msgs sent

RESULTS

Duration : 19.63s
Clients OK/Fail : 15/35
Messages Sent : 300
Throughput : 15.28 msg/s

Metric Join Latency Msg Round-Trip

AVG 6242.05 ms 6390.33 ms
P50 7030.77 ms 6436.83 ms
P95 7416.51 ms 6755.16 ms
P99 7416.51 ms 6755.16 ms
MIN 683.00 ms 6034.13 ms
MAX 7416.51 ms 6755.16 ms
Count 15 16

```

Table 1: Performance Benchmarking Comparison Between Server A and Server B

Category	Metric	Server A	Server B
General Metrics	Duration	25.57 s	19.63 s
	Clients (OK / Fail)	19 / 31	15 / 35
	Messages Sent	380	300
	Throughput	14.86 msg/s	15.28 msg/s
Join Latency (ms)	Average	3660.66	6242.05
	P50	1068.49	7030.77
	P95	8765.80	7416.51
	P99	8765.80	7416.51
	Min	289.47	683.00
	Max	8765.80	7416.51
	Count	19	15
Msg Round-Trip (ms)	Average	5328.78	6390.33
	P50	5894.92	6436.83
	P95	8844.92	6755.16
	P99	9467.83	6755.16
	Min	127.97	6034.13
	Max	9556.39	6755.16
	Count	175	16

8 Detailed Comparative Analysis

8.1 Connection Success and Throughput

- **Client Connection Capacity:** Server A (1 backend instance) successfully accepted and completed connections for **19 out of 50 clients** (38% success rate), whereas Server B (3 backend instances behind the load balancer) completed only **15 out of 50 clients** (30% success rate).
- **System Throughput:** Server B registered a slightly higher raw throughput rate (15.28 msg/s) than Server A (14.86 msg/s), primarily because Server B aborted failed client sessions earlier, resulting in a shorter total test duration (19.63 s vs. 25.57 s).

8.2 Join Latency Dynamics

- **Median Latency (P_{50}):** Server A established client connections significantly faster at the 50th percentile (1068.49 ms) compared to Server B (7030.77 ms), demonstrating a $\approx 6.5\times$ speed advantage for median connection establishment.
- **Tail Latency (P_{95}/P_{99}):** Under peak load, Server B plateaued its maximum join latency at 7416.51 ms, whereas Server A experienced higher tail spikes reaching 8765.80 ms. This indicates that distributing connections across 3 servers capped worst-case join delays, but introduced significant initial routing overhead at the load balancer level.

8.3 Message Round-Trip and Cluster Synchronization

- **Message Completion Rate:** Server A successfully delivered **175 round-trip messages** out of 380 sent (46.05% delivery rate). In contrast, Server B completed only **16 round-trip messages** out of 300 sent (5.33% delivery rate), pointing to a critical bottleneck in multi-node message fan-out or sticky-session routing.
- **Latency Floor vs. Consistency:** Server A achieved direct local socket performance with a minimum latency of 127.97 ms, but suffered high variation ($P_{99} = 9467.83$ ms). Server B exhibited a very high baseline delay (6034.13 ms minimum), but maintained tighter consistency across higher percentiles ($P_{99} = 6755.16$ ms).

9 Code of load balancer

10 LLM Usage Policy

Large Language Models (LLMs) were utilized as assistive tools during the development of this project in the following specific areas:

- **User Interface Design:** Assisting in prototyping, structuring, and refining UI/UX components.
- **Backend Development:** Providing syntax guidance, code optimization, and troubleshooting support for Go implementation within the load balancer.

```

from aiohttp import web, ClientSession, WSMsgType
Ctrl-C for Agent

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [LB] %(levelname)-7s %(message)s",
    datefmt="%H:%M:%S",
)
log = logging.getLogger("load_balancer")

# Keep at most this many latency samples in memory (oldest are dropped)
MAX_LATENCY_SAMPLES = 50_000

# Metrics collector

class MetricsCollector:
    """Thread-safe-ish (single event-loop) metrics store."""

    def __init__(self, backends: list[str]):
        self.backends = backends
        self.start_time = time.time()

        # Counters
        self.total_http = 0
        self.total_ws = 0
        self.active_ws = 0
        self.total_errors = 0
        self.total_messages = 0

        # Per-backend counters
        self.http_per_backend: dict[str, int] = defaultdict(int)
        self.ws_per_backend: dict[str, int] = defaultdict(int)
        self.active_ws_per_backend: dict[str, int] = defaultdict(int)
        self.errors_per_backend: dict[str, int] = defaultdict(int)
        self.messages_per_backend: dict[str, int] = defaultdict(int)

        # Latency samples: list of (timestamp, latency_ms)
        self.http_latencies: list[tuple[float, float]] = []
        self.ws_handshake_latencies: list[tuple[float, float]] = []
        self.msg_relay_latencies: list[tuple[float, float]] = []

# ---- Helpers ----
@staticmethod
def _percentile_stats(samples: list[tuple[float, float]]) -> dict:
    if not samples:
        return {"count": 0, "avg_ms": 0, "min_ms": 0, "max_ms": 0,
                "p50_ms": 0, "p95_ms": 0, "p99_ms": 0}
    vals = sorted(v for _, v in samples)
    n = len(vals)
    return {
        "count": n,
        "avg_ms": round(statistics.mean(vals), 3),
        "min_ms": round(vals[0], 3),
        "max_ms": round(vals[-1], 3),
        "p50_ms": round(vals[n // 2], 3),
    }

```

Figure 3:

```

60 class MetricsCollector:
61     @staticmethod
62     def _percentile_stats(samples: list[tuple[float, float]]) -> dict:
63         if not samples:
64             return {"count": 0, "avg_ms": 0, "min_ms": 0, "max_ms": 0,
65                     "p50_ms": 0, "p95_ms": 0, "p99_ms": 0}
66         vals = sorted(v for _, v in samples)
67         n = len(vals)
68         return {
69             "count": n,
70             "avg_ms": round(statistics.mean(vals), 3),
71             "min_ms": round(vals[0], 3),
72             "max_ms": round(vals[-1], 3),
73             "p50_ms": round(vals[n // 2], 3),
74             "p95_ms": round(vals[int(n * 0.95)] if n >= 20 else vals[-1], 3),
75             "p99_ms": round(vals[int(n * 0.99)] if n >= 100 else vals[-1], 3),
76         }
77
78     def _trim(self, lst: list) -> None:
79         if len(lst) > MAX_LATENCY_SAMPLES:
80             del lst[:len(lst) - MAX_LATENCY_SAMPLES]
81
82     def record_http(self, backend: str, latency_ms: float) -> None:
83         self.total_http += 1
84         self.http_per_backend[backend] += 1
85         self.http_latencies.append((time.time(), latency_ms))
86         self._trim(self.http_latencies)
87
88     def record_ws_open(self, backend: str, handshake_ms: float) -> None:
89         self.total_ws += 1
90         self.active_ws += 1
91         self.ws_per_backend[backend] += 1
92         self.active_ws_per_backend[backend] += 1
93         self.ws_handshake_latencies.append((time.time(), handshake_ms))
94         self._trim(self.ws_handshake_latencies)
95
96     def record_ws_close(self, backend: str) -> None:
97         self.active_ws -= 1
98         self.active_ws_per_backend[backend] = max(
99             0, self.active_ws_per_backend[backend] - 1
100         )
101
102     def record_message(self, backend: str, relay_ms: float) -> None:
103         self.total_messages += 1
104         self.messages_per_backend[backend] += 1
105         self.msg_relay_latencies.append((time.time(), relay_ms))
106         self._trim(self.msg_relay_latencies)
107
108     def record_error(self, backend: str) -> None:
109         self.total_errors += 1
110         self.errors_per_backend[backend] += 1
111
112     def snapshot(self) -> dict:
113         uptime = time.time() - self.start_time
114         total_reqs = self.total_http + self.total_ws

```

Figure 4:

```
1 class LoadBalancer:
2     def _next_backend(self) -> str:
3         backend = self.backends[self._rr_index % len(self.backends)]
4         self._rr_index += 1
5         return backend
6
7     # WebSocket proxy
8     async def handle_ws(self, request: web.Request) -> web.WebSocketResponse:
9         backend = self._next_backend()
10        ws_url = (
11            backend.replace("http://", "ws://")
12            .replace("https://", "wss://")
13            .rstrip("/")
14            + "/ws"
15        )
16        # Accept incoming client WebSocket
17        ws_client = web.WebSocketResponse(
18            heartbeat=30.0,
19            max_msg_size=4 * 1024 * 1024,
20        )
21        await ws_client.prepare(request)
22
23        log.info("WS %s -> %s", request.remote, backend)
24
25        try:
26            t0 = time.time()
27            async with self.session.ws_connect(
28                ws_url,
29                heartbeat=30.0,
30                max_msg_size=4 * 1024 * 1024,
31            ) as ws_backend:
32                handshake_ms = (time.time() - t0) * 1000
33                self.metrics.record_ws_open(backend, handshake_ms)
34
35                # --- bidirectional relay ---
36                async def client_to_backend():
37                    try:
38                        async for msg in ws_client:
39                            if msg.type == WSMsgType.TEXT:
40                                t = time.time()
41                                await ws_backend.send_str(msg.data)
42                                self.metrics.record_message(
43                                    backend, (time.time() - t) * 1000
44                                )
45                            elif msg.type == WSMsgType.BINARY:
46                                await ws_backend.send_bytes(msg.data)
47                                self.metrics.record_message(backend, 0)
48                            elif msg.type in (WSMsgType.CLOSE, WSMsgType.CLOSING,
49                                           WSMsgType.CLOSED, WSMsgType.ERROR):
50                                break
51                    except Exception:
52                        pass
53
54                async def backend_to_client():
55                    try:
```

Figure 5:

```

class LoadBalancer:
    async def handle_ws(self, request: web.Request) -> web.WebSocketResponse:
        async def backend_to_client():
            break
        except Exception:
            pass

        # Run both directions; when either finishes, cancel the other
        task_c2b = asyncio.create_task(client_to_backend())
        task_b2c = asyncio.create_task(backend_to_client())
        _done, pending = await asyncio.wait(
            [task_c2b, task_b2c],
            return_when=asyncio.FIRST_COMPLETED,
        )
        for t in pending:
            t.cancel()

        except Exception as exc:
            log.error("WS proxy error (%s): %s", backend, exc)
            self.metrics.record_error(backend)
        finally:
            self.metrics.record_ws_close(backend)
            log.info(
                "WS %s-20s <- %s (active: %d)",
                request.remote, backend, self.metrics.active_ws,
            )

        return ws_client

# ---- HTTP reverse proxy ----
async def handle_http(self, request: web.Request) -> web.Response:
    backend = self._next_backend()
    target = backend.rstrip("/") + request.path_qs

    try:
        t0 = time.time()
        async with self.session.request(
            method=request.method,
            url=target,
            headers={
                k: v
                for k, v in request.headers.items()
                if k.lower() not in ("host", "transfer-encoding")
            },
            data=await request.read(),
            allow_redirects=False,
        ) as resp:
            latency_ms = (time.time() - t0) * 1000
            self.metrics.record_http(backend, latency_ms)

            body = await resp.read()
            headers = {
                k: v
                for k, v in resp.headers.items()
                if k.lower()

```

Figure 6:

```

# -----
def create_app(backends: list[str]) -> web.Application:
    app = web.Application()
    lb = LoadBalancer(backends)
    app["lb"] = lb

    async def on_startup(_app: web.Application) -> None:
        await lb.start()
        log.info("=" * 60)
        log.info("  LOAD BALANCER STARTED")
        log.info("  Algorithm : Round-Robin")
        log.info("  Backends  : %d", len(backends))
        for b in backends:
            log.info("    %s", b)
        log.info("  Dashboard : http://localhost:<port>/lb/metrics")
        log.info("=" * 60)

    async def on_shutdown(_app: web.Application) -> None:
        await lb.stop()
        log.info("Load balancer stopped.")

    app.on_startup.append(on_startup)
    app.on_shutdown.append(on_shutdown)

    # Catch-all route
    app.router.add_route("*", "/{path_info:.+}", router)

    return app

def main() -> None:
    parser = argparse.ArgumentParser(
        description="WebSocket-aware Round-Robin Load Balancer"
    )
    parser.add_argument(
        "--backends",
        required=True,
        help="Comma-separated backend URLs, e.g. http://10.1.75.79:3201,http://10.1.75.79:3202",
    )
    parser.add_argument(
        "--port",
        type=int,
        default=8080,
        help="Port the load balancer listens on (default: 8080)",
    )
    parser.add_argument(
        "--host",
        default="0.0.0.0",
        help="Host to bind to (default: 0.0.0.0)",
    )
    args = parser.parse_args()

    backends = [b.strip().rstrip("/") for b in args.backends.split(",") if b.strip()]
    if not backends:
        parser.error("At least one backend URL is required.")

```

View 1 edited file requirements.txt Alt+L >

Figure 7: